Exercícios Avaliativos - LinkedList

Os alunos deverão modificar as classes, adicionando novas funcionalidades enquanto mantêm o comportamento original da estrutura de dados.

📌 Exercício 1: Histórico de Remoções

Desafio: Modifique a LinkedList para armazenar um histórico dos elementos removidos.

🧠 **Regras:**

1. Crie uma estrutura #removedHistory para armazenar os valores dos nós removidos.
2. Sempre que um valor for removido (remove()), adicione-o ao histórico.
3. Crie um método history() que retorne os elementos removidos.
4. Teste com inserções e remoções, e exiba o histórico final.

```
Lista atual: ( [0]: Item1, [1]: Item2, [2]: Item3 ), count: 3
Histórico de remoções: Nenhuma remoção ainda.
Lista após remoção: ( [0]: Item1, [1]: Item3 ), count: 2
Histórico de remoções: [ Item2 ]
```

📌 Exercício 2: Reversão da Lista

Desafio: Crie um método reverse() que **inverta a ordem dos nós** da lista encadeada.

🧠 **Regras:**

1. Após chamar reverse(), o primeiro elemento vira o último e vice-versa.
2. A lista original é modificada — sem criar outra lista.
3. Teste a funcionalidade com print() antes e depois de inverter.

```
Lista antes de inverter: ( [0]: Item1, [1]: Item2, [2]: Item3 ), count: 3
Lista após inversão: ( [0]: Item3, [1]: Item2, [2]: Item1 ), count: 3
```

📌 Exercício 3: Contagem de Ocorrências (REPETIDOS)

Desafio: Crie um método countOccurrences(val) que conta quantas vezes um valor aparece na lista.

🧠 **Regras:**

1. O método deve percorrer toda a lista.
2. Retorna um número representando a quantidade de vezes que val aparece.
3. Teste com inserções repetidas de um mesmo valor.

```
Lista atual: ( [0]: Item1, [1]: Item2, [2]: Item1, [3]: Item3, [4]: Item1 ),
count: 5
Ocorrências de 'Item1': 3
Ocorrências de 'Item2': 1
```

📌 Exercício 4: Remoção de Todas as Ocorrências

Desafio: Crie um método removeAll(val) que remove **todas as ocorrências** de um valor na lista.

🧠 Regras:

1. Percorra a lista e remova cada ocorrência de val.
2. Ajuste os ponteiros corretamente após cada remoção.
3. Teste com uma lista contendo vários valores repetidos.

```
Lista antes da remoção de 'Item1': ( [0]: Item1, [1]: Item2, [2]: Item1, [3]:
Item3, [4]: Item1 ), count: 5
Lista após remoção de 'Item1': ( [0]: Item2, [1]: Item3 ), count: 2
```

📌 Exercício 5: Transformar Lista em Array

Desafio: Crie um método toArray() que transforma a lista em um array comum ([]).

🧠 Regras:

1. O método deve retornar um array com os dados dos nós na ordem correta.
2. Utilize esse array para facilitar comparações e visualização.
3. Teste imprimindo o array retornado.

```
Lista atual como array: [ 'Item1', 'Item2', 'Item3' ]
```